CSE 214: Principles of Database Systems

Winter 2026

Instructor: Nikolaos (Nikos) Tziavelis ntziavel@ucsc.edu
Time & Place: Tuesdays & Thursdays 05:20PM-06:55PM, Oakes Acad 106
Office Hours: To be announced in Canvas

Welcome to CSE 214! This course is an introduction to the principles of database systems, i.e., the theory that underlies the systems that we use for data management. The topics we will study cover classic database theory, as well as more recent research areas. Contrary to a systems-oriented course, we will pay less attention to the low-level details of any specific database system, and focus more on higher-level concerns, including algorithms, computational complexity, and the properties of query languages. You will have the opportunity to apply the theoretical concepts through homeworks and a project. The course encourages active participation, self-study, and project-based learning.

Learning Objectives:

- Get an overview of theoretical problems that arise in databases, and some recent research trends.
- Learn how to analyze algorithms for query processing and how to think about their optimality.
- Become familiar with different query languages and their associated complexity.
- Explore how database theory translates into query optimization in practice.

Tentative Schedule:

Topic	Number of lectures
Review of the relational data model, relational algebra & calculus, SQL	3
The query evaluation problem and its complexity	2
Conjunctive queries, acyclicity, and the Yannakakis algorithm	3
Size bounds for joins, worst-case optimal joins, query decompositions	4
Parallel query processing	3
Datalog and recursive queries	2
Database dependencies and the chase procedure	2

Prerequisites: The course requires basic familiarity with discrete math and complexity theory (e.g., complexity classes such as PTIME and NP and the notion of completeness). CSE 201 is listed as a formal prerequisite, but can be waived if the student can demonstrate similar experience. Basic knowledge of databases and SQL is also required, as typically taught in an introductory undergraduate course.

Course Grading:

- Project: 50%
- Homework: 30%
- Scribes: 15%
- Participation: 5%

Project: You will work on a project related to database theory that will introduce you to research in this area. Projects should be done in teams of two, but working alone or in teams of three is also a possibility (with corresponding expectations). Examples of projects include an implementation of a theoretical algorithm together with a performance analysis, an extension of an algorithm found in a research paper to cover new cases, or a systematic exploration of a new research idea. In the last case, it is not necessary to "solve" the problem to do well; outlining a list of attempts, partial progress, or a substantial literature survey can also result in a strong project score. You can come up with their own project, or meet with me to discuss project ideas.

The project involves the following milestones (dates are tentative and subject to change):

- Project proposal
  - By the end of 5th week
  - A short description of what the project will be about
- Intermediate progress report
  - By the end of 8th week
  - An extension of the proposal to include the progress made by that point.
  - This counts as **10%** of the final grade.
- Project presentation
  - During finals week
  - You will present the project in class and answer any questions that come up.
  - This counts as **15%** of the final grade.
- Final report
  - By the end of finals week
  - The final report should be written like a typical research paper, addressing any feedback from the presentation or the earlier report.
  - This counts as **25%** of the final grade.

Homework: There will be three homework assignments. In addition to theory questions, they will also involve a programming part, where you will write queries in SQL so that you better understand the algorithms we discuss in class. You will execute the queries using an open-source database system. The homework counts as **30%** of the final grade.

Spotlight Presentation: Once during the quarter, you will pick a concept from previously covered material (can be anything from the lecture slides), illustrate it with your own slide deck (can be 1 slide), and present it in class in 1-3 minutes. To ensure presentations are distributed throughout the quarter, you will need to sign up for a slot and present something that we covered in the previous 4 lectures. The goal is to explain the concept in your own way as if you were teaching it, so be creative! Time yourself to make sure that your presentation can be completed in 1-3 minutes. Examples:

- Present a definition visually.
- Illustrate the worst-case behavior of an algorithm with a minimal example.
- Find some new information about a topic we only touched on briefly.

This presentation counts as **15%** of the final grade.

Participation: You are expected to attend class and ask/answer questions. Participation counts as **5%** of the final grade.

Late submission policy: There are 3 "grace days" (self-granted extensions) that can be used freely. Instructor-granted extensions are only considered in exceptional situations. A job interview or the exam of another course does not qualify as an exceptional situation.

Academic Integrity: The Official University Policy on Academic Integrity for Graduate Students can be found at https://www.ucsc.edu/academics/academic-integrity/

Disability Accommodations: UC Santa Cruz is committed to creating an academic environment that supports its diverse student body. If you are a student with a disability who requires accommodations to achieve equal access in this course, please affiliate with the DRC. I encourage all students to benefit from learning more about DRC services to contact DRC by phone at 831-459-2089 or by email at drc@ucsc.edu. For students already affiliated, make sure that you have requested Academic Access Letters, where you intend to use accommodations. You can also request to meet privately with me during my office hours or by appointment, as soon as possible. I would like us to discuss how we can implement your accommodations in this course to ensure your access and full engagement in this course.

Recommended Textbook & Resources: The standard reference is the "Alice" book:

- Foundations of Databases. Abiteboul, Hull, Vianu.
  (You can find a copy for personal use here: http://webdam.inria.fr/Alice/)

There is also a new book in the works. While still incomplete, it is very useful and relevant for the course:

- Principles of Databases. Marcelo Arenas, Pablo Barcelo, Leonid Libkin, Wim Martens, Andreas Pieris. https://github.com/pdm-book/community

Some of the content is drawn from the following research papers:

Query Complexity

- The Complexity of Relational Query Languages. Vardi. STOC 1982.
- Algorithms for acyclic database schemes. Yannakakis. VLDB 1981.
- Size bounds and query plans for relational joins. Atserias, Grohe, Marx. FOCS 2008.
- Hypertree Decompositions and Tractable Queries, Gottlob, Leone, Scarcello. JCSS 2002.
- Leapfrog Triejoin: a worst-case optimal join algorithm. Veldhuizen. ICDT 2014.
- Skew Strikes Back: New Developments in the Theory of Join Algorithms. Ngo, Re, Rudra. SIGMOD Record 2013.
- FAQ: Questions Asked Frequently. Abo Khamis, Ngo, Rudra. PODS 2016.

Datalog

- What You Always Wanted to Know About Datalog (And Never Dared to Ask). Ceri, Gottlob, Tanca. TKDE 1989.

Parallel Query Processing

- MapReduce: simplified data processing on large clusters. Dean, Ghemawat. OSDI 2004.
- MapReduce and parallel DBMSs: friends or foes?. Stonebraker et al. CACM 2010.
- Optimizing Joins in a Map-Reduce Environment. Afrati, Ullman. EDBT 2010.
- A Guide to Formal Analysis of Join Processing in Massively Parallel Systems. Koutris, Suciu. SIGMOD Record 2016.

Acknowledgements: The lecture slides partially rely on content by Phokion Kolaitis who taught previous offerings of the course. They also draw inspiration and content from other courses by Wolfgang Gatterbauer, Paris Koutris, Mirek Riedewald, and Dan Olteanu.